Security-Relevant Evolution of a GitSkills

SWE 380 / CSC 580 Group 8 Project Report

Jim Prantzalos Kyle Cantrell Vaisnavii Mohanraj Christian Heiney

Fall 2026

Abstract

[TODO: In approximately 150-250 words, summarize the research problem, research question, dataset, method, principal results, and implications. Complete this section after the analysis is finished.]

Introduction

[TODO: Introduce AI-agent skills, GitSkills, and the broader software-engineering context of the project.]

Background and Context

GitSkills artifacts can contain instructions that direct AI agents to execute commands, access or modify files, communicate with external services, invoke scripts, or handle credentials. These behaviors are often legitimate and necessary, but they are security-relevant because carrying them out could affect a system, its data, or external resources.

Skills may also be modified over time or reused across repositories. During that process, security-relevant instructions may be added, removed, or altered. A modified skill could introduce command execution, file-system access, network access, script use, credential handling, or another consequential behavior that was not present in an earlier version or related source artifact.

Motivation

Studying these changes can provide evidence about how security-relevant behavior evolves within GitSkills and whether reuse or modification introduces new risk signals. A reproducible analysis can also help developers and reviewers identify changes that may warrant closer security review.

This topic also has personal relevance to the team because two members are pursuing cybersecurity concentrations. It provides an opportunity to apply security-focused coursework to an emerging software-engineering problem while developing practical experience in threat identification, static analysis, and responsible interpretation of security evidence.

Expected Contribution

This project is expected to produce:

1. A documented taxonomy of security-relevant instruction patterns, including command execution, file-system operations, external network access, script use, and credential-related instructions.
2. A reproducible, rule-based static-analysis pipeline for detecting those patterns in GitSkills artifacts without executing untrusted instructions or scripts.
3. A reproducible method for identifying and comparing related skill artifacts across versions or reuse relationships.
4. An analysis showing whether security-relevant instruction patterns are introduced, removed, retained, or modified between earlier or source artifacts and later or related artifacts.
5. Empirical measurements describing the prevalence of each defined pattern and the frequency with which new patterns appear during observed skill evolution or reuse.
6. A validation and reporting approach that documents detection limitations, relationship uncertainty, ambiguous cases, and false positives while distinguishing observable risk signals from confirmed vulnerabilities or malicious behavior.

Together, these contributions will provide a repeatable method for examining both the presence and evolution of security-relevant instructions in GitSkills artifacts.

Report Organization

[TODO: Briefly describe how the remainder of the report is organized.]

Research Question and Expectations

Selected Topic

This project investigates Question 4, **Skill security and supply-chain risk**, from the MSR 2027 Mining Challenge-inspired project outline.

Research Question

> **Do modified or reused skills introduce command execution, file-system access, network access, or other risky behavior that was absent from an earlier version or source artifact?**

Expectations

[TODO: Describe the patterns the team expects to observe. Clearly distinguish expectations from findings.]

Operational Definitions

These definitions specify how the research concepts are represented and measured by the tools. Scanner rules operationalize security-relevant text patterns; relationship rules operationalize candidate relatedness and direction. Both are needed to interpret an introduced signal. The detailed algorithms and the current 13-rule catalog appear in Method and Implementation.

Artifact Occurrence and Content Variant

An artifact is one collected skill-file occurrence at a repository and path within the dataset snapshot. The local `artifact_id` identifies that occurrence in the saved DuckDB build. Occurrences with identical skill-file hashes share a skill-content variant but remain distinct artifacts. A bundle variant additionally accounts for known sibling-file content; missing sibling information is not equivalent to an empty bundle.

Candidate Family and Related Artifacts

A candidate family comprises qualifying occurrences with the same exact skill name, including differing description subgroups. Shared names define the search space, not verified reuse or ancestry. Distinct skill variants are considered related when they satisfy the configured shingle-similarity policy. Bundle states sharing the same skill content can also be compared for sibling changes. A connected similarity cluster can contain endpoints that do not themselves pass the direct pairwise test.

Base Artifact

The base is the source endpoint of an ordered comparison. For a selected family relationship, it is the inferred predecessor supported by the implemented direction procedure: resolved declared provenance, observed chronology or equivalent-peer chronology, a repository-creation boundary, or asymmetric containment. The evidence basis is retained because these alternatives provide different kinds of support. A base is not necessarily the earliest published artifact or the original author's version.

For `scan_diff`, the base is simply the first input path supplied by the caller. For the extraction notebook, it is the immediately preceding occurrence within the same name group in view order. These input conventions do not independently establish lineage.

Derived Artifact

The derived artifact is the target endpoint compared against the base. In the selected family graph, it is the inferred child according to the recorded direction evidence. In a direct file comparison, it is the second input path; in the extraction notebook, it is the next occurrence in the adjacent pair. Calling an endpoint derived does not prove direct copying, historical descent, or malicious modification. Direction-unknown relationships are retained as unresolved and excluded from directional family scans.

Security-Relevant Instruction Pattern

A security-relevant instruction pattern is text matched by a rule in the active static-scanner catalog. The current categories are command execution, network access, file-system access, credential access, external code execution, and system modification. A rule is a specific detector within a category: for example, EXT-002 detects recognized package-installation patterns within external code execution, while NET-001 detects HTTP/HTTPS URLs within network access.

A rule match records the rule ID, assigned category, and available location or excerpt evidence. A category is present when at least one rule assigned to it matches. Skill scans normally exclude leading YAML frontmatter; sibling text is scanned without that skill-specific exclusion. Matched content is never executed by the scanner. A match can occur in legitimate instructions or documentation and does not establish that an action occurred, that a resource was sensitive, or that a vulnerability exists.

Introduced Risk Signal

An introduced signal is measured relative to both endpoints, using the same rule definitions and a comparable scan scope. The study distinguishes:

Measure	Operational condition	Example
Introduced rule	A rule has zero matches in the base and at least one in the derived artifact	Package-install matches change from 0 to 1.
Introduced capability category	No rule in a category matches the base, while at least one matches the derived artifact	External code execution is absent from the base profile and present in the derived profile.
Increased existing signal	A rule already matches the base and its count increases	Package-install matches change from 1 to 3, producing a delta of +2.

The current comparator's JSON field `introduced` records category introductions. Its `rule_deltas` field records rules with changed match counts, including increases and decreases; it is not a list limited to newly appearing rules. A new rule can appear within a category already represented in the base, so rule introduction and category introduction are separate measures. More matches for an existing rule need not introduce either.

These terms describe changes in detected text, not proven increases in harm. Family-level review can also flag matches in new or changed destination sibling content without an increased aggregate count; such findings must be labeled by scan surface and should not automatically be called introduced capabilities. Missing required content makes the relevant coverage incomplete rather than negative.

Match Count, Delta, and Retained Signals

A rule's match count is the number of non-overlapping matches retained within that rule after its pattern matches are deduplicated. Matches across different rules remain separate, so the total is not a count of unique actions. The delta is the derived count minus the base count. Positive and negative values indicate more or fewer matches, respectively; zero indicates equal counts, not necessarily identical instructions.

A rule or category is retained when present in both endpoint profiles and removed when present only in the base. Retention concerns presence, so a retained rule can still have a positive or negative count delta. A rule omitted from `rule_deltas` can be present with equal counts on both sides. These definitions do not imply that the current output serializes separate retained and removed sets.

Modified or Reused Skill

For this study, a potentially modified or reused skill is an occurrence connected to another through the implemented candidate and relationship procedure. Exact content identity supports an observation of shared content; qualifying similarity supports candidate relatedness; available direction evidence supports an ordered comparison. None alone proves the mechanism of transfer or authorial intent.

Modification can concern the skill body, bundled sibling content, or both. Shared content in separate occurrences does not itself demonstrate modification. The main directional analysis uses selected predecessor relationships, while equivalent and unresolved relationships remain separately reported. The notebook's adjacent comparisons serve extraction and scanner testing and must not be counted as independently verified reuse relationships.

Dataset and Data Preparation

This section describes the dataset source, study scope, data schema, sampling strategy, eligibility filters, preprocessing steps, and safety controls used to construct the analysis-ready data. The detailed algorithms and implementation pipeline are described in the Method and Implementation section.

Dataset Source and Acquisition

The source dataset for this study is `agent_skills_release.db`, the GitSkills database provided for the MSR 2027 Mining Challenge. The source population consists of all GitSkills artifact records contained in the database.

[TODO: Add the approved dataset access instructions, release or snapshot identifier, file integrity information, and the date on which the database was obtained. Explain that the full database is not committed to the project repository and identify the expected local storage location.]

Dataset Scope

The study distinguishes three populations:

1. **Source population:** All GitSkills artifact records in `agent_skills_release.db`.
2. **Analyzable artifact population:** Records from the source population that contain stored skill content that the static analyzer can process.
3. **Eligible comparison population:** Ordered base-derived comparisons that can be extracted from the database and satisfy the eligibility criteria defined below.

Artifacts without analyzable content or without an identifiable, ordered relationship remain part of the source population, but they are not part of the eligible comparison population. The pipeline will report the number of records and candidate relationships excluded at each stage and the reasons for exclusion.

The local selection audit below verifies 3,797,117 source occurrences and 943,257 retained candidates. Candidate membership is narrower than the population of all records with stored text. Counts of unique analyzed content SHAs, supported relationships, completed scans, and final eligible directed comparisons still require the corresponding pipeline run; they cannot be inferred from the candidate-table row count.

Candidate-Population Reduction

From the Source Release to Comparison Candidates

The source release contains approximately 3.8 million discovered file occurrences, often rounded to four million. These are artifact occurrences rather than four million distinct skill contents or four million eligible comparisons. The setup in `data/README.md` imports the SQLite release into a separate DuckDB database, then invokes `sql/create_analysis_tables.sql` to materialize `artifact_groupings`. The selection leaves the source `artifacts` table intact. It narrows the candidate population for reuse analysis rather than deleting source records or drawing a random sample.

The following is the executable candidate-selection statement from `sql/create_analysis_tables.sql`. The preceding `DROP TABLE IF EXISTS artifact_groupings` rebuilds only the derived table; it is not a source-data exclusion criterion.

```
CREATE TABLE artifact_groupings AS
WITH filtered AS (
    SELECT
        a.id AS artifact_id,
        a.repo_id,
        a.name,
        regexp_replace(
            lower(trim(a.description)),
            '\s+',
            ' ',
            'g'
        ) AS normalized_description
    FROM artifacts AS a
    WHERE a.frontmatter_valid = 1
      AND a.body_chars > 50
      AND a.content IS NOT NULL
      AND a.name IS NOT NULL
      AND a.description IS NOT NULL
),
eligible_names AS (
    SELECT
        name
    FROM filtered
    GROUP BY name
    HAVING COUNT(*) >= 2
),
groups AS (
    SELECT
        f.name,
        f.normalized_description,
        COUNT(*) AS artifact_group_size
    FROM filtered AS f
    JOIN eligible_names AS e ON e.name = f.name
    GROUP BY
        f.name,
        f.normalized_description
),
group_ids AS (
    SELECT
        row_number() OVER ()::BIGINT AS id,
        name,
        normalized_description,
        artifact_group_size
    FROM groups
)
SELECT
    g.id,
    f.artifact_id,
    f.repo_id,
    f.name,
    f.normalized_description,
```

```
    g.artifact_group_size,
    NULL::BIGINT AS sibling_file_count,
    NULL::VARCHAR AS sibling_content_sha
FROM filtered AS f
JOIN eligible_names AS e ON e.name = f.name
JOIN group_ids AS g ON g.name = f.name
 AND g.normalized_description = f.normalized_description;
```

Preparation Workflow

The activity diagram summarizes the implemented candidate reduction and two separate analysis objectives. The branch represents a chosen workflow, not concurrent execution or a claim that the planned population census has been completed. Counts describe the local selection audit; the Sprint 1 path uses a purposeful subset.

Content and Metadata Filters

The `filtered` common table expression (CTE) selects each artifact's identifier, repository identifier, exact name, and normalized description. A CTE names an intermediate query result so later stages can refer to the same selection logic. The `WHERE` conditions are combined with `AND`, so every condition must be satisfied; a missing value in either numeric condition does not pass the filter.

SQL condition	What it retains	Methodological rationale and cost
`a.frontmatter_valid = 1`	Records marked as having valid frontmatter	Provides structured skill metadata for candidate construction. It can exclude useful instructions with missing or malformed frontmatter; it is a format-quality criterion, not a security judgment.
`a.body_chars > 50`	Bodies with strictly more than 50 characters	Screens out very short or near-empty bodies before comparison. Fifty characters is a pragmatic screening threshold, not a validated minimum for meaningful behavior or a guarantee of enough five-token shingles. Short legitimate skills can be excluded.
`a.content IS NOT NULL`	Occurrences with stored text available	Similarity and static scanning require analyzable text. Missing stored content cannot be interpreted as empty or safe content; availability can reflect the dataset's representative-enrichment strategy.

SQL condition	What it retains	Methodological rationale and cost
`a.name IS NOT NULL`	Records with a parsed name	The implemented candidate loader uses exact-name families. Without a name, an artifact cannot enter that search space; related artifacts lacking metadata will be missed.
`a.description IS NOT NULL`	Records with a parsed description	Allows the query to construct description-based groups consistently. This is required by the current grouping design, even though exact-name family loading does not require descriptions to match. Missing descriptions can therefore exclude otherwise useful same-name candidates.

These tests check non-nullness, not nonempty trimmed strings. The query does not explicitly reject blank names or descriptions, require a particular filename or directory, restrict natural language, require commit dates, retain only `dedup_primary` rows, or filter for security-rule matches. In particular, selecting records with valid frontmatter is different from scanning frontmatter: metadata supports candidate selection and provenance, while behavioral scans normally exclude that metadata.

Description Normalization

`trim(a.description)` removes leading and trailing whitespace; `lower(...)` removes case distinctions; and `regexp_replace(..., '\s+', ' ', 'g')` replaces every run of whitespace with a single space. The rationale is to avoid splitting description groups because of formatting or capitalization alone. This does not remove punctuation, translate prose, or establish semantic equivalence. The name is copied unchanged: matching remains exact and case-sensitive under the query's ordinary string equality.

Repeated Names and Description Groups

`eligible_names` groups the already-filtered rows by exact `name` and applies `HAVING COUNT(*) >= 2`. At least two qualifying occurrences are needed to form a within-name comparison candidate. The count is taken after the content and metadata filters, so a name with multiple raw occurrences can still be excluded if only one survives those conditions. The threshold counts occurrences, not distinct file SHAs, repositories, or revisions; two copies can qualify without demonstrating modification.

The `groups` CTE joins those eligible names back to the filtered rows and groups by both `f.name` and `f.normalized_description`. `COUNT(*) AS artifact_group_size` records the size of each description subgroup. There is no second minimum-size test at this level. A unique description remains eligible when its name is shared by another qualifying artifact. This preserves description changes that may accompany reuse; requiring two identical name-and-description occurrences would discard those variants prematurely. Family analysis can subsequently compare across description groups within the same exact name, as recorded in RDR-003.

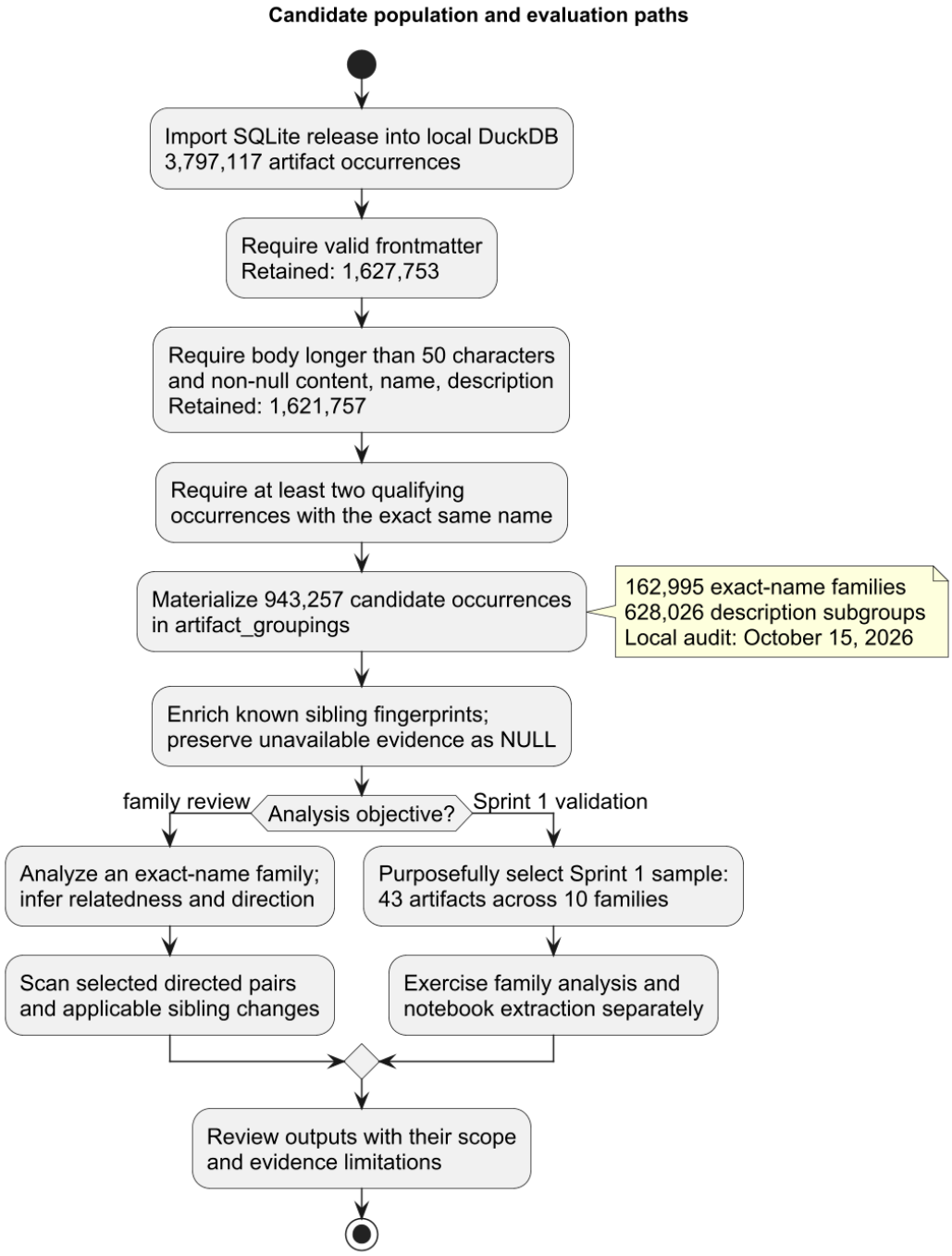


Figure 1: Activity diagram of candidate preparation and the family-review and Sprint 1 validation paths.

Artifact, Family, and Group Identity

The grouping design provides reproducible membership rules and explicit links between artifact occurrences, candidate families, and description subgroups. These are different identities. An artifact ID answers which collected occurrence is being analyzed; a family key answers which same-name candidate population is being searched; and a group ID answers which name-and-description subgroup contains the occurrence. None of these identifiers proves common ancestry.

Entity	Identifier used by the current implementation	Meaning and scope
Artifact occurrence	`artifacts.id`, copied into `artifact_groupings.artifact_id`	A surrogate integer assigned during DuckDB import; identifies one occurrence within that saved build. The original logical occurrence key is `(repo_full_name, path)` within the dataset snapshot.
Repository	`repos.id`, retained as `repo_id`	A local join identifier; the retained repository name provides source context.
Exact-name candidate family	`artifact_groupings.name`	All qualifying occurrences sharing that exact name, across description groups. The current schema does not assign a separate numeric family ID.
Name-and-description subgroup	`artifact_groupings.id`, called `group_id` by the Python loader	One distinct `(name, normalized_description)` combination. Its ID repeats across all member rows and is not an individual-artifact identifier.
Skill-content variant	`artifacts.file_sha`	A content identifier shared by byte-identical skill files; does not uniquely identify their repository occurrences or bundled resources.

`artifact_id` is created before candidate selection. The final query copies it from `artifacts.id` rather than renumbering retained artifacts, so the same occurrence can be traced through filtering, family analysis, graph endpoints, and scanner comparisons. By contrast, `artifact_groupings.id` is assigned after grouping and intentionally appears on multiple membership rows. Identifying a group by `id` alone therefore cannot identify its individual members. The existing local audit found one distinct `artifact_id` for every retained row, but the schema does not enforce primary-key or uniqueness constraints; this property should be checked after rebuilding or changing joins.

Consider the following synthetic example; these labels and values are not dataset findings:

Local artifact ID	Source occurrence	Exact family name	Normalized description	Local group ID	Skill SHA
101	Repository X, path P	`example-skill`	`inspect status`	7	H1
102	Repository Y, path Q	`example-skill`	`inspect status`	7	H1
103	Repository Z, path R	`example-skill`	`inspect and report status`	8	H2

This represents three occurrences, one exact-name family, two description groups, and two skill-content variants. Artifacts 101 and 102 remain distinct despite sharing content and a group ID. Artifact 103 remains in the family despite its different description. Later similarity and direction analysis determine whether these are useful ordered comparisons; membership alone supplies no such conclusion. A similarity cluster is also a later analysis result, not another name for the SQL group. Its membership and numbering depend on the loaded candidates and similarity policy.

Deterministic Membership Versus Build-Local Numbers

For identical source values and fixed filtering and normalization rules, an occurrence's exact-name family and name-and-description subgroup are determined by those values. This is the reproducible part of the design. The numeric IDs have a narrower guarantee: both the import query and the grouping query use `row_number() OVER ()` without an explicit `ORDER BY`. Consequently, neither artifact IDs nor group IDs are guaranteed to receive the same values after an independent rebuild, even if observed builds have happened to agree. Persisted numbers remain usable when reopening the same unchanged database.

The team independently downloaded `agent_skills_release.db` and ran the project SQL queries on their own machines. Team members reported accurately reproducing the results across these independent local setups. This provides empirical evidence that the documented preparation and grouping workflow was repeatable in the environments exercised by the group. Observed agreement is distinct from a guarantee that unordered numeric ID assignments remain stable in every rebuild or future dataset release.

To reconcile independent builds, retain the dataset release or snapshot identity together with the original occurrence key `(repo_full_name, path)`. Use exact `name` for the candidate-family key and `(name, normalized_description)` for the subgroup key, with the normalization definition recorded. A file SHA can additionally check content identity but cannot replace occurrence identity because the same content may appear in several repositories. Across dataset releases, the same repository and path may contain different content, so the snapshot context is essential.

The current schema does not implement a persistent registry or hash-derived identifier for families and groups. Adding an explicit ordering to row numbering could make assignment repeatable for a fixed input, but additions or removals can still shift later numbers. A future persistent identity scheme would need to account for that behavior before promising stable numeric IDs across releases. For the current study, preserve the database build or retain the original logical keys and a verified mapping; do not treat an unqualified numeric ID as a global identifier.

Final Joins and Sibling Placeholders

`group_ids` assigns a `BIGINT` identifier to each name-and-description group using `row_number() OVER ()`. The group identifier organizes related membership rows; it does not identify an individual

artifact. Because this window has no `ORDER BY`, its numbering must not be assumed stable across rebuilds. Source linkage uses the retained `artifact_id`, with repository context retained through `repo_id`.

The final joins retain only eligible names and attach each filtered artifact to its matching name-and-description group. The result has one membership row per retained source artifact; grouping does not collapse occurrences to one representative or prove lineage. The stored `artifact_group_size` describes a subgroup, not the total exact-name family.

`NULL::BIGINT AS sibling_file_count` and `NULL::VARCHAR AS sibling_content_sha` create typed placeholders. They do not filter out artifacts lacking sibling data, and null does not mean no sibling files. The next setup script, `sql/update_artifact_groupings.sql`, populates these fields only when composition was fetched, was not truncated, and every sibling file has a content hash. Known empty bundles receive count zero and the fingerprint of the empty string; incomplete composition remains unknown. The fingerprint uses a sorted multiset of sibling-file hashes and ignores paths and directory entries. This enrichment distinguishes known bundle states without requiring all candidates to have complete sibling evidence.

Selection Counts and Interpretation

A read-only audit of the prepared local DuckDB database on October 15, 2026 produced the following counts using the current selection conditions:

Cumulative selection stage	Retained artifacts	Excluded at this step
All source artifact occurrences	3,797,117	—
`frontmatter_valid = 1`	1,627,753	2,169,364
`body_chars > 50`	1,621,757	5,996
`content IS NOT NULL`	1,621,757	0
`name IS NOT NULL`	1,621,757	0
`description IS NOT NULL`	1,621,757	0
Exact name occurs in at least two qualifying rows	943,257	678,500

The final selection retains **943,257 of 3,797,117 artifacts** (24.84 percent) and excludes 2,853,860 (75.16 percent). This is a reduction in the number of artifact occurrences, not a measured reduction in pairwise comparisons or a count of distinct skill contents. The materialized `artifact_groupings` table independently contains 943,257 rows and 943,257 distinct artifact IDs, organized into 162,995 exact-name families and 628,026 name-and-normalized-description groups. Its row and family totals agree with the recomputed selection totals; agreement of totals alone is not a full membership-equivalence check.

The frontmatter test removes most occurrences in the displayed sequence. The body-length condition then removes 5,996, and the repeated-name requirement removes 678,500 otherwise qualifying occurrences. The three non-null checks remove no additional rows after the preceding conditions in this local database. That observation does not establish that missing content, names, or descriptions never occur in the source; they may already have been removed by earlier conditions. Keeping these explicit checks documents and enforces the prerequisites across rebuilds and other releases.

These are current local counts, not a claim that every retained artifact has been scanned or validated. The earlier exploratory figure of 1,146,188 artifacts recorded in RDR-001 predates later selection refinements and should not replace the current query's result. A new database build or revised query requires a new audit. The source release identifier and integrity information still need to be recorded before treating these counts as a fully frozen release experiment.

Counts at each line are cumulative in the order used for explanation, not a claim about the database engine's physical evaluation order. They are also not independent exclusion totals: an artifact can fail more than one condition, and attributing exclusions to a particular step depends on the displayed order. The following read-only SQL reproduces the cumulative audit and summarizes the materialized candidate table without changing either source or analysis tables. Open the prepared database in DuckDB and execute both statements:

```
duckdb -readonly /c/data/duckdb/agent_skills_release.db
```

```
-- Read-only audit of the selection in create_analysis_tables.sql.
-- Rows are cumulative in documentation order, not SQL execution order.
-- No source or analysis tables are changed.
WITH stages AS (
    SELECT
        name,
        frontmatter_valid = 1 AS valid_frontmatter,
        body_chars > 50 AS long_enough_body,
        content IS NOT NULL AS has_content,
        name IS NOT NULL AS has_name,
        description IS NOT NULL AS has_description
    FROM artifacts
),
eligible_names AS (
    SELECT name, COUNT(*) AS artifact_count
    FROM stages
    WHERE valid_frontmatter
      AND long_enough_body
      AND has_content
      AND has_name
      AND has_description
    GROUP BY name
    HAVING COUNT(*) >= 2
),
counts AS (
    SELECT 0 AS step, 'Source artifacts' AS selection_stage,
           COUNT(*) AS retained_artifacts
    FROM stages
    UNION ALL
    SELECT 1, 'Valid frontmatter', COUNT(*)
    FROM stages WHERE valid_frontmatter
    UNION ALL
    SELECT 2, 'Body longer than 50 characters', COUNT(*)
    FROM stages WHERE valid_frontmatter AND long_enough_body
    UNION ALL
    SELECT 3, 'Stored content present', COUNT(*)
    FROM stages WHERE valid_frontmatter AND long_enough_body AND has_content
```

```
    UNION ALL
    SELECT 4, 'Name present', COUNT(*)
    FROM stages
    WHERE valid_frontmatter AND long_enough_body AND has_content AND has_name
    UNION ALL
    SELECT 5, 'Description present', COUNT(*)
    FROM stages
    WHERE valid_frontmatter AND long_enough_body AND has_content
      AND has_name AND has_description
    UNION ALL
    SELECT 6, 'Name shared by at least two qualifying artifacts',
           coalesce(SUM(artifact_count), 0)::BIGINT
    FROM eligible_names
)
SELECT
    step,
    selection_stage,
    retained_artifacts,
    lag(retained_artifacts) OVER (ORDER BY step) - retained_artifacts
        AS excluded_at_this_step
FROM counts
ORDER BY step;

-- Independent summary of the already materialized candidate table.
SELECT
    COUNT(*) AS retained_artifacts,
    COUNT(DISTINCT artifact_id) AS distinct_artifact_ids,
    COUNT(DISTINCT name) AS exact_name_families,
    COUNT(DISTINCT id) AS name_description_groups
FROM artifact_groupings;
```

This population reduction precedes shingle similarity, direction inference, and scanner comparison. It also precedes the purposeful Sprint 1 selection of 43 artifacts. The retained candidate count must not be reported as a number of proven relatives, eligible directed comparisons, security findings, or manually validated cases. Exact-name selection can miss renamed relatives, and content/metadata requirements favor better-enriched artifacts. These selection effects delimit any later population-level claims.

Relevant Schema

The analysis uses data at four related levels: artifact, relationship, scanner profile, and ordered comparison. The field lists below describe the broader target schema, not a claim that every field is present in the current Sprint 1 reports. Exact source-table and column names will be verified against the implemented extraction pipeline and documented in `DATA_DICTIONARY.md`. Current wrapper reports record endpoint artifact IDs, relationship evidence, introduced-capability flags, total match counts, and nonzero rule-count deltas.

Artifact Data

For both base and derived artifacts, the prepared data retains the available fields needed for identification, analysis, and traceability:

- artifact identifier;
- repository identifier;
- artifact path;
- declared skill name, when available;
- content SHA;
- stored content or a safe reference to the analyzed content; and
- available timestamp, commit, or other ordering metadata used by the relationship method.

Relationship Data

Each candidate or eligible relationship records:

- comparison identifier;
- base artifact identifier;
- derived artifact identifier;
- relationship type;
- same-repository or cross-repository status;
- relationship-detection method;
- evidence supporting the relationship;
- evidence supporting base-derived ordering;
- relationship confidence or review status, if used by the implemented method; and
- eligibility or exclusion status and reason.

Scanner-Profile Data

For each analyzed artifact, the prepared scanner profile contains:

- matched rule identifiers;
- matched security-relevant categories;
- match counts by rule and category;
- total matched-rule and matched-category counts;
- match locations or other evidence required for validation and safe reporting; and
- scanner and rule-catalog versions.

Comparison Data

For each eligible ordered comparison, the prepared analysis table contains:

- introduced, removed, and retained rule identifiers;
- introduced, removed, and retained categories;
- counts of introduced rules and categories;
- indicators for at least one introduced rule and at least one introduced category; and
- the overall comparison result under the documented threat model.

Rule-level and category-level fields are both retained. A derived artifact may introduce a new rule within a category that was already present in the base artifact; that change is visible at the rule level even when the set of categories does not change.

Unit of Analysis

The primary unit of analysis is one ordered comparison between two related, content-bearing skill artifacts:

- **Base artifact:** The inferred source endpoint, according to the supported direction evidence.

- **Derived artifact:** The inferred target endpoint being compared with the base; this designation does not prove historical descent.

The ordering is represented as:

base artifact -> derived artifact

One unique content SHA is the secondary processing unit. The pipeline may scan identical content once and reuse its profile, but SHA-based deduplication is only a computational optimization. Artifact occurrences, repository occurrences, and comparison relationships remain available after cached profiles are joined back to the complete extracted data.

Inclusion and Exclusion Criteria

An ordered comparison is eligible when all of the following conditions are satisfied:

1. Both artifacts contain stored content that the scanner can analyze.
2. The extraction pipeline identifies a relationship using the project's documented relationship method.
3. Available evidence supports the base-derived ordering.
4. Both artifacts can be processed with the same scanner version and rule-catalog version.

A candidate comparison is excluded when one or more of these conditions are not satisfied. The pipeline retains the exclusion status and reason so that filtering is auditable. Uncertain or invalid relationships may be reviewed and excluded according to the documented relationship method.

[TODO: Add any implemented content-format, parsing, duplication, missing-value, or relationship-confidence filters and the exact exclusion codes used by the pipeline.]

Data Preparation

The preparation process converts the source database into analysis-ready artifact and comparison data. It will:

1. extract artifact content and traceability metadata from the relevant source tables;
2. identify candidate relationships and retain the evidence used to establish each relationship;
3. determine base-derived ordering from the available evidence;
4. apply the documented inclusion and exclusion criteria;
5. normalize content only as required by the scanner while preserving the original content or a safe reference to it;
6. identify unique content SHAs for cached scanning without discarding artifact occurrences;
7. attach scanner profiles to artifact records; and
8. join base and derived profiles into the ordered comparison table.

[TODO: Document the implemented text decoding, line-ending normalization, parsing, missing-content handling, and any other cleaning transformations. Distinguish transformations used for matching from the original content retained for traceability.]

Population Analysis and Sampling

Planned Automated Census

The automated analysis will attempt a census of the eligible comparison population rather than select a statistical sample. Every eligible base-derived comparison extracted from `agent_skills_release.db` will be processed by the comparison pipeline.

Smaller reproducible datasets may be used while developing and debugging the pipeline. These development datasets are test inputs, not the research sample, and will not replace the final population-level analysis.

Planned Manual Validation Sample

For the broader evaluation, the team plans to manually review a stratified sample of at least 100 eligible base-derived comparisons from the automated results. This future plan is separate from the 17 directed comparisons in the current Sprint 1 reports:

- at least 50 comparisons for which the scanner reports one or more newly introduced rules or categories; and
- at least 50 comparisons for which the scanner reports no newly introduced rules or categories.

Within those two groups, the sample will represent the relationship types and scanner categories present in the results. Selection will use a recorded random seed after the strata are defined. If a stratum contains fewer cases than its allocation, all cases in that stratum will be reviewed and the remaining allocation will be redistributed using a documented procedure.

The sample may be expanded if early review identifies unstable rules, poorly represented relationship types, or categories that require additional evidence. The manual-review procedure, labels, metrics, and error analysis belong in the Evaluation and Validation section.

For each sampled comparison, reviewers will record:

- whether the relationship and base-derived ordering are supported by the retained evidence;
- whether each reported match corresponds to the text identified by the scanner;
- whether the assigned categories are appropriate;
- whether a reported introduction was absent from the base and present in the derived artifact;
- whether an unflagged comparison contains a relevant introduction missed by the current rules; and
- whether the case is a true positive, false positive, false negative, true negative, or ambiguous under the documented threat model.

These records will support confirmed and rejected relationship counts, ordering disagreements, classification counts, precision and any other justified validation metrics, category- or relationship-specific breakdowns where sample size permits, and representative failure modes. The Evaluation and Validation section will define the final calculation and interpretation of those measures.

Sprint 1 Sampling Strategy

Sprint 1 uses a small, purposeful tool-validation sample of **43 artifacts across 10 exact-name families**. The ten saved analysis reports contain **17 directed comparisons**. These totals describe the current tool output, not a representative population sample or completed manual validation of every comparison.

The earlier design combined purposeful cases with 75 random artifact seeds and family expansion, producing approximately 1,400 artifact-selection rows. It was too large for immediate manual validation. The revised selection aimed for approximately 50 artifacts or fewer and has no separate random component. The earlier sample may remain useful for later work; its reconstruction checks do not validate the revised selection.

The Evaluation and Validation section explains how these artifacts exercised candidate loading, lineage analysis, scanning, and extraction. It distinguishes the 17 selected directed family comparisons from the extraction notebook's 33 adjacent comparisons. The two counts describe different pairing procedures applied to the sample, not competing estimates of its size.

Selection Logic and Family Scope Families were selected to exercise supported direction evidence, ambiguous relationships, cases without inferred relationships, and positive, negative, or unchanged scanner counts. Other cases distinguish additional matches within an existing capability from newly detected capabilities and expose scanner coverage or normalization limitations.

`artifact_groupings` groups artifacts by exact name and normalized description. The current `analyze_family` tool loads all qualifying artifacts sharing an exact name, including multiple description groups. Some screening queries compare only within a name-and-description group, so their screening scope is narrower than the family-analysis scope. Shared metadata establishes candidate context, not proven lineage.

The selection queries screened for package-installation count variation, supported provenance-field references, and variation in additional scanner rules. SQL patterns were selection heuristics rather than validated findings. Candidate families were checked through the Python workflow, and selected comparisons were inspected manually in Mergely. Searching ended because further screening was time-consuming, not because the sample was exhaustive or statistically optimized. The query rationale and RDR-006 record the selection decisions.

Observed Coverage and Limits The saved Sprint 1 reports documented nonzero count deltas in 10 of the then-current 12 rules. EXT-001 and SYS-001 lacked confirmed delta examples in that historical selection. The current implementation registers 13 rules, including FS-003 for file reading; the older coverage total must not be treated as coverage of the current catalog. Rule presence, a count change, and an introduced capability are different outcomes; none alone establishes increased or decreased risk.

The Sprint 1 wrapper scanned selected directed relationships rather than every possible artifact pair and compared only `SKILL.md` content. These limits describe the saved Sprint 1 output. The current `scan_family` implementation also reviews available sibling content for applicable changes and reports incomplete coverage. Ambiguous relationships remain excluded from directional scanning; a family with no directed comparisons does not establish an absence of matches. Historical skill-only results must not be interpreted as findings about sibling safety.

Selected manual inspections exposed uneven command coverage and a line-ending sensitivity bug. The Sprint 1 wrapper was corrected to normalize line endings before scanning, and that comparison was retained for regression checking in the earlier workflow. This historical correction is not evidence that every current input path applies identical normalization. Other inspected cases illustrate limitations in declared-source resolution and exact-name candidate generation. These observations do not constitute manual validation of every comparison.

Sample View and Reproducibility The revised selection uses the 43 `artifact_id` values recorded for the team's saved database build. These values identify occurrences consistently within that build; the unordered import numbering does not guarantee the same mapping in a separate build. The script `sql/create_sprint1_samples_view.sql` embeds the selected IDs, selects matching `artifact_groupings` rows, and joins them to `artifacts` and `repos` to create or replace `sample_artifacts`. The CSV at `data/sample-population/sample_population.csv` separately records selected IDs; the view does not read that file.

The view stores the selection query rather than copying artifact records into separate sample tables. Its ID list fixes the selection criteria, while returned values reflect the underlying `artifacts` table when queried. The current view exposes grouping membership, selected source fields including content and paths, and repository metadata through those joins. It is not an anonymized reporting view.

The script's final count query should return 43 if every selected occurrence remains in the candidate table and resolves through the artifact and repository joins. The saved family-report totals do not demonstrate that this query has been run successfully. Reproducibility checks should reconcile the embedded IDs with the recorded list, confirm that every ID resolves, and compare reconstructed membership in both directions. A matching row count alone is insufficient to establish identical membership.

The saved ID list supports reconstruction in the original build. Reconstructing the intended occurrences in an independent build also requires verifying the ID-to-source mapping through the original logical keys. The dataset version, database build, selection SQL, scanner rules, and normalization settings must therefore be recorded. Earlier frozen-table validation applies only to the earlier sample.

Data Ethics and Safety

All artifact content is treated as untrusted data. The project performs static analysis only and will not execute scripts, commands, or instructions contained in the dataset.

The pipeline and reporting process will:

- avoid executing or importing untrusted artifact code;
- avoid exposing credentials, tokens, or other sensitive strings that may appear in artifact content;
- keep validation evidence local and omit source text and identifying values from exported reports under the current output constraints;
- distinguish detected risk signals from proof of malicious intent, vulnerability, exploitability, or real-world harm; and
- report aggregate findings and generalized observations without naming selected artifacts or reproducing source content.

Current orchestration reports identify endpoints by artifact ID, omit text diffs, and serialize scanner comparisons with `verbose=False`. These measures limit exported content but do not guarantee anonymity or resolve licensing questions. Artifact IDs can be linked to source records by someone with access to the dataset.

[TODO: Document the implemented redaction, access-control, retention, and safe-example-selection procedures.]

Method and Implementation

Method Overview

The study measures whether modified or reused GitSkills introduce security-relevant behavior that was absent from an earlier version or source artifact. For each eligible ordered comparison, the method scans the base and derived artifacts with the same versioned rule catalog, constructs a rule and category profile for each artifact, and measures which signals were introduced, removed, or retained in the derived artifact.

The current implementation combines exact-name candidate loading, token-shingle similarity, evidence-based direction inference, and a static scanner with 13 registered rules across six categories. The executable tools support individual file comparisons and family-level review. A completed population census and statistical validation are separate research tasks; the existence of these tools does not establish that those tasks have been completed. Rule definitions, dependency versions, code revision, and analysis settings must be recorded for each reported run.

Analysis Algorithm

Artifact Profiling

Let (R(a)) be the set of rule identifiers detected in artifact (a), and let (C(a)) be the set of security-relevant categories represented by those rules. For each artifact, the static analyzer records the matched rules, categories, counts, and match evidence required for validation.

Base-Derived Comparison

For an eligible ordered pair with base artifact (b) and derived artifact (d), the comparison stage computes:

$$R_{introduced} = R(d) \setminus R(b)$$

$$R_{removed} = R(b) \setminus R(d)$$

$$R_{retained} = R(b) \cap R(d)$$

These sets describe the study's intended rule-presence measures. The current comparator directly emits introduced-category flags and nonzero rule-count deltas; it does not serialize separate introduced, removed, and retained rule sets. Those sets can be derived from complete endpoint profiles.

The same set operations define category-level measures:

$$C_{introduced} = C(d) \setminus C(b), \quad C_{removed} = C(b) \setminus C(d), \quad C_{retained} = C(b) \cap C(d)$$

A comparison has a rule-level introduction when $|R_{introduced}| > 0$, and it has a category-level introduction when $|C_{introduced}| > 0$. Both measures are retained because a new rule may appear within a category already present in the base artifact.

Match-Count Changes and Introduced Capabilities

Let (n_r(a)) be the number of matches for rule (r) in artifact (a). The implemented comparator calculates:

$$\Delta_r = n_r(d) - n_r(b)$$

A positive delta means more matched text for that rule, a negative delta means fewer matches, and zero means an unchanged count. The `rule_deltas` output includes only rules whose counts differ. Its omission of a rule therefore does not establish that the rule was absent from both artifacts.

Three outcomes must be distinguished:

Outcome	Condition	Interpretation
New rule presence	Base count is zero and derived count is positive	A previously undetected rule appears.
Introduced capability	A category is absent from the base profile and present in the derived profile	A previously undetected category appears.
Increased existing signal	Both counts are positive and the derived count is larger	More matches occur without necessarily introducing a rule or category.

For example, increasing package-install matches from one to three produces a delta of +2. If external code execution was already detected in the base, this increase does not introduce that capability. Conversely, a new rule may appear inside an existing category. Total counts are sums of rule matches, not counts of unique actions: one instruction may match multiple rules. Equal totals can conceal different matched text or offsetting changes.

Outcome Computation

The planned population-level primary outcome is the proportion of eligible comparisons in which the derived artifact contains at least one detected security-relevant signal that was absent from the base:

$$\text{Rule introduction rate} = \frac{\text{eligible comparisons with at least one introduced rule}}{\text{eligible comparisons analyzed}}$$

$$\text{Category introduction rate} = \frac{\text{eligible comparisons with at least one introduced category}}{\text{eligible comparisons analyzed}}$$

Secondary outputs may include:

- the count and percentage of comparisons introducing each rule or category;
- distributions of introduced-rule and introduced-category counts;
- counts and percentages of comparisons with removed signals;
- counts and percentages with retained signals but no introduced signals;
- counts and percentages with no detected profile change;
- introduction rates by relationship type;
- introduction rates for same-repository and cross-repository comparisons; and

- the numbers of distinct artifacts, content SHAs, and repositories represented.

Every reported denominator will identify whether it represents comparison occurrences, unique artifact relationships, or unique content-SHA pairs.

Threat Model and Rule Catalog

The static analyzer is intended to identify security-relevant instructions or behaviors such as executable commands, URLs, scripts, file operations, credential-related instructions, and other categories adopted by the final threat model. A rule match is a reviewable signal, not a conclusion that an artifact is malicious or unsafe.

Each rule in the catalog should define:

- a stable rule identifier;
- rule name and description;
- higher-level security-relevant category;
- detection logic or pattern reference;
- severity or review priority, if adopted by the threat model;
- known false-positive conditions;
- positive and negative examples; and
- rule-catalog version.

Category Rationale

The categories operationalize the trust boundaries described in `THREAT_MODEL.md`: instructions can cause an agent to execute programs, communicate externally, access local data, handle secrets, obtain executable dependencies, or alter its host environment. These are consequential capabilities even when required by a legitimate task. The catalog favors inspectable technical indicators that can be matched without running untrusted content. This is a rationale for the current scope, not evidence that the catalog detects every security-relevant behavior.

Category	Registered rules and indicators	Reason for inclusion	Interpretation limit
Command execution	CMD-001: shell-language code fences; CMD-002: recognized command invocations	Instructions can launch interpreters and executable programs.	A fence or command reference can be explanatory documentation.
Network access	NET-001: HTTP/HTTPS URLs; NET-002: recognized network-client usage	Instructions can contact external resources or retrieve data.	A URL alone does not show a request or data transfer.
File-system access	FS-001: file commands; FS-002: writing APIs; FS-003: reading commands/APIs	Instructions can read or modify local data.	Detection does not establish sensitive targets or harmful changes.

Category	Registered rules and indicators	Reason for inclusion	Interpretation limit
Credential access	CRED-001: secret-related terms; CRED-002: credential-file references	Instructions can involve authentication material and secret-bearing resources.	A reference is not evidence that a real secret was exposed.
External code execution	EXT-001: download-to-shell pipelines; EXT-002: package installation	Instructions can introduce externally supplied executable content.	Legitimate dependency installation is common; source trust is not verified.
System modification	SYS-001: privilege elevation; SYS-002: system-changing commands	Instructions can affect permissions and host configuration.	Administrative workflows may legitimately require these commands.

The executable registry is `src/gitskills/rules/registry.py`; category-specific modules contain the patterns. Rules have stable IDs, category assignments, descriptions, and detection logic. The current registry does not itself establish a separately released rule-catalog version or calibrated severity score. Record the code revision alongside `uv.lock` and run settings when identifying the catalog used.

For illustration, a synthetic instruction such as `curl https://example.com/install.sh | bash` can match URL, network-client, command, and download-and-execute rules. Overlapping categories are deliberate perspectives on the same text, not independent harmful actions. A synthetic `pip install example-package` instruction explains package installation without establishing that the package is harmful. Unrecognized syntax, indirect execution, and obfuscation remain potential sources of missed findings.

Static-Analysis Algorithm

The scanner processes artifact content as text and produces rule-level and category-level profiles without executing the content. Each match retains sufficient location or excerpt evidence for validation and safe reporting. The base and derived artifacts in one comparison are always scanned with the same scanner and rule-catalog versions.

`SkillAnalyzer` masks leading YAML frontmatter by default, preserving line locations while excluding metadata from behavioral matching. Frontmatter can contain repository URLs and permission descriptions that do not constitute body instructions. Provenance extraction uses selected frontmatter fields separately for relationship analysis. Sibling resources are scanned without this skill-specific frontmatter exclusion.

Each registered `RegexRule` applies its patterns using case-insensitive, multiline matching. Within one rule, candidate matches are sorted by position and longer matches at the same position are preferred; overlapping spans are discarded. Matches from different rules remain separate. Each match records the rule ID, category, message, line number, and matched text. The capability profile is the set of categories represented by at least one match. Duplicate rule IDs are rejected when constructing the analyzer.

This is a text-pattern engine rather than a shell parser, interpreter, or execution monitor. It does not infer whether an instruction ran, whether an endpoint was reachable, or whether a resource was

sensitive. Verbose command output supplies evidence for review, with credential-related excerpts redacted by the reporting layer. Read failures and database failures must be distinguished from successful scans with no matches.

Relationship and Ordering Algorithm

The relationship stage identifies related artifacts, determines which artifact is the base and which is derived, and records the evidence supporting both decisions. A comparison proceeds only when the relationship and ordering satisfy the eligibility criteria in the Dataset and Data Preparation section.

Candidate Families and File States

`DuckDBFamilyRepository` loads qualifying artifacts from `artifact_groupings` by exact skill name, across normalized-description groups. A name match creates a candidate search space; it does not establish a common author or copying relationship. This restriction can miss renamed relatives and include independently developed skills.

An artifact is one collected occurrence. A skill variant groups occurrences sharing the raw skill-file SHA. A bundle variant combines that skill state with a known sibling-content fingerprint. Unknown sibling state is conservatively kept separate by occurrence rather than treated as an empty or matching bundle. Distinguishing these units prevents identical skill text from hiding changes in associated scripts or resources.

Similarity and Relatedness

For each distinct skill variant, the implementation masks frontmatter, tokenizes the body, lowercases tokens, and constructs the set of distinct contiguous five-token shingles. Technical structures such as URLs, paths, flags, and dotted identifiers are preserved where the tokenizer recognizes them. Repetition of an identical shingle does not increase its weight.

For shingle sets (A) and (B):

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}, \qquad K(A \to B) = \frac{|A \cap B|}{|A|}$$

Jaccard measures symmetric overlap; containment measures how much of one representation appears in the other. Empty-set comparisons receive zero similarity. The current exploratory defaults require maximum directional containment of at least 0.80, Jaccard of at least 0.30, and at least five shared shingles. All three conditions must pass for distinct skill variants. Bundle variants with the same raw skill SHA can be compared without a distinct-skill similarity test.

Consider a synthetic example with 100 shingles in A, 250 in B, and 90 shared. Containment A to B is 0.90, containment B to A is 0.36, and Jaccard is 90/260, approximately 0.346. The pair passes the defaults. Requiring Jaccard as well as containment prevents a small text embedded in a much larger file from qualifying solely because all its shingles were retained. These are exploratory heuristics, not validated probabilities of copying; threshold calibration and sensitivity analysis remain evaluation work.

Qualifying similarity relationships form connected clusters. If A relates to B and B relates to C, all three can occupy one cluster even if A and C fail the direct pairwise test. Scores of 1.0 describe the shingle representation and do not necessarily establish byte-identical original files.

Direction Evidence and Uncertainty

The implementation considers direction only after relatedness is established. It first resolves supported declared-source fields (`derived_from`, `upstream_skill`, `source_repo`, and `upstream_source`) against the loaded family. Conflicting declarations preserve an ambiguous relationship. A resolved source provides the strongest direction basis; an unresolved reference does not show that the source is absent from the full dataset.

Equivalent states without a resolved declared source remain undirected. For other relationships, the decision order is:

1. Compare distinct effective observed chronology points. Available evidence can come from an equivalent bundle peer; reports distinguish that proxy from direct chronology.
2. If chronology does not resolve the pair, check whether an observed state predates the other repository's creation boundary. Repository creation is a boundary, not the skill's creation date.
3. Otherwise, use asymmetric containment when the difference between the two containment scores is at least 0.10. Infer direction from the more-contained state toward the other state.
4. Preserve the relationship as ambiguous if no basis resolves direction.

`first_commit_at` describes observed history at the current path, which can omit history before a move or rename. Containment supplies a structural hypothesis, not a verified temporal sequence. Consequently, the terms parent and child denote inferred base and derived endpoints throughout this report, not proven direct historical descent.

Graph Selection and Change Classification

Candidate directed edges are ranked by evidence basis: declared source, direct chronology, equivalent-peer chronology, repository boundary, and containment. Tie-breaking considers evidence count, temporal proximity where applicable, similarity, shared shingles, and deterministic endpoint identity. The selector retains at most one predecessor per target bundle and rejects candidates that would create a cycle. Other supported directed relationships remain available as additional comparisons; a related pair need not become a selected parent edge.

Relationships are classified as skill-only, siblings-only, skill-and-siblings, equivalent, or unknown according to skill similarity/state and available sibling fingerprints. A root candidate is a node with no selected predecessor, not a verified original author or original publication.

The state diagram provides a conceptual model of a candidate bundle pair's classification. The code computes these outcomes rather than maintaining a mutable state field. Declared-source decisions take precedence over fallback evidence; equivalent states without a resolved source remain undirected. Predecessor selection considers the surrounding graph, not just the isolated pair.

Ambiguous relationships are retained as uncertainty and excluded from directional family scanning. They must not be counted as comparisons with no security signals.

Tools and Implementation Architecture

The implementation is organized around four functional components:

1. **Extractor:** Reads the relevant records and metadata from `agent_skills_release.db`.
2. **Relationship builder:** Produces candidate ordered base-derived relationships and eligibility evidence.
3. **Static analyzer:** Applies the versioned rule catalog to unique artifact content and creates rule and category profiles.

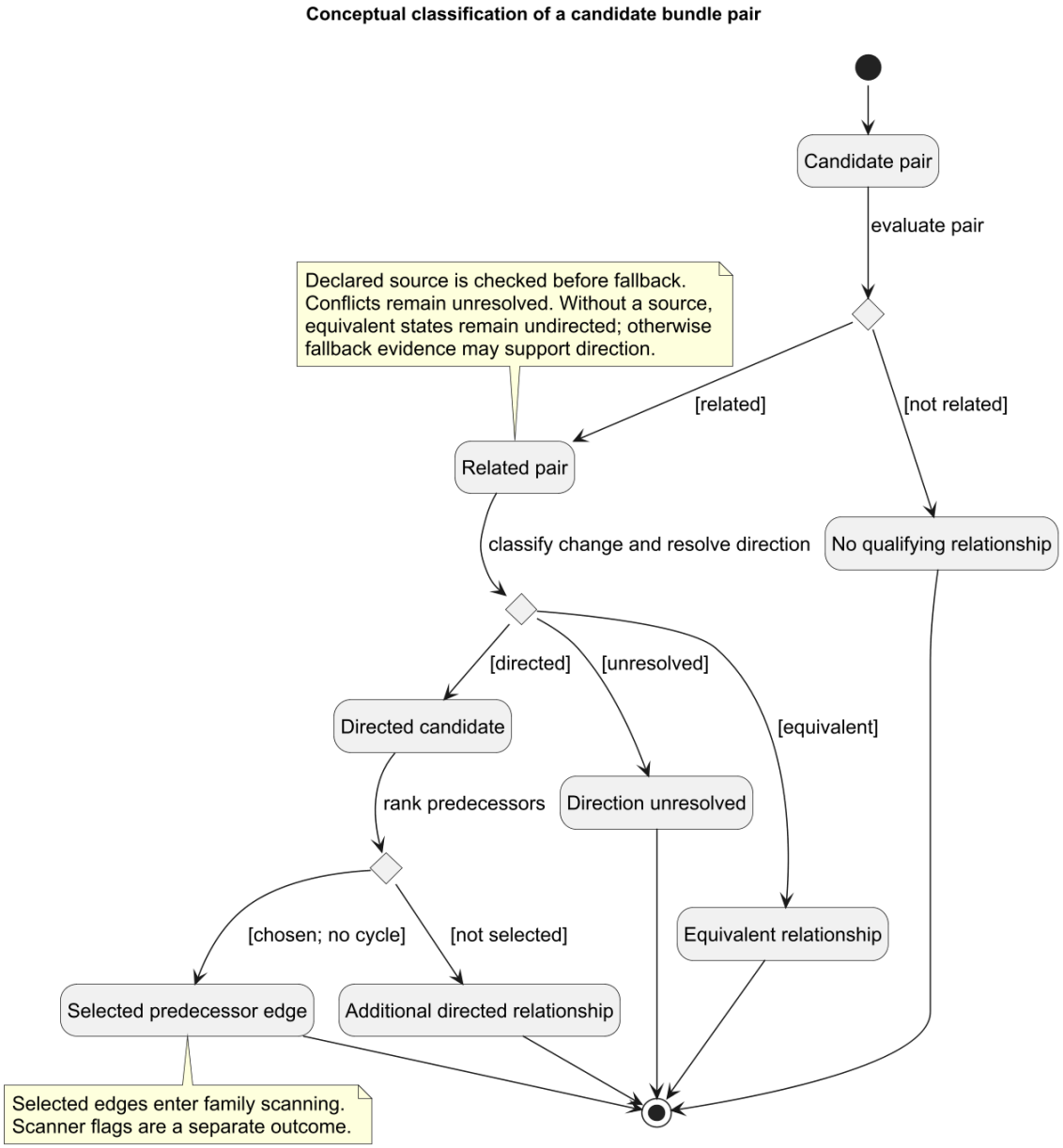


Figure 2: Conceptual UML state diagram for relationship classification. Scanner flags are separate from lineage outcomes.

4. **Comparator and reporter:** Joins profiles to eligible relationships, computes changes and aggregate measures, and writes reproducible analysis artifacts.

The project uses Python 3.12 or newer, DuckDB for analysis queries, and dependencies recorded in `pyproject.toml` and `uv.lock`. `FamilyAnalyzer` implements relationship construction; `SkillAnalyzer` and `compare_results` implement text scanning and paired comparison; `FamilyRiskAnalyzer` orchestrates directed-family review. Tests use pytest. SkillTrace is a FastAPI application for inspecting generated family reports, content, diffs, and pair scans. The report sources are Markdown combined through `report/pandoc.yaml`.

Entry point	Input and purpose	Scan scope
`analyze_family`	Exact family name and DuckDB; infer and report relationships	Relationship analysis; no security scan
`scan_diff`	Two local paths, base first; compare rule counts and capabilities	The two supplied skill bodies
`scan_family`	Exact family name and DuckDB; review selected directed edges	Skill bodies and available sibling content for applicable changes
SkillTrace	Saved analysis reports plus DuckDB; inspect graph and content	Its pair scan compares selected skill bodies

`scan_family` flags positive skill-rule count deltas. Its sibling review considers aggregate count increases and rule matches in new or changed destination content; such a finding can exist without a positive aggregate delta. Missing sibling content or SHA evidence is recorded as incomplete coverage. Neither an unflagged pair nor an incomplete scan establishes safety.

Family-Scan Component Interactions

The sequence diagram follows the successful `scan_family` execution path. The CLI loads both artifacts and sibling records before passing them to `FamilyRiskAnalyzer`; that analyzer invokes `FamilyAnalyzer` and reviews the selected directed edges. Calls to the scanner can reuse cached results. Sibling review is conditional on the classified change type and records unavailable evidence separately from negative findings. Argument, database, and output-write failure paths are omitted for readability.

End-to-End Pipeline

The broader population-analysis target is:

1. Load the required source tables from `agent_skills_release.db`.
2. Extract analyzable artifact content and traceability metadata.
3. Identify candidate relationships and determine base-derived ordering.
4. Apply eligibility filters and record exclusions.
5. Identify unique content SHAs.
6. Scan each unique SHA with the configured scanner and rule catalog.
7. Join cached profiles back to all applicable artifact occurrences.
8. Compare base and derived profiles to calculate introduced, removed, and retained rules and categories.

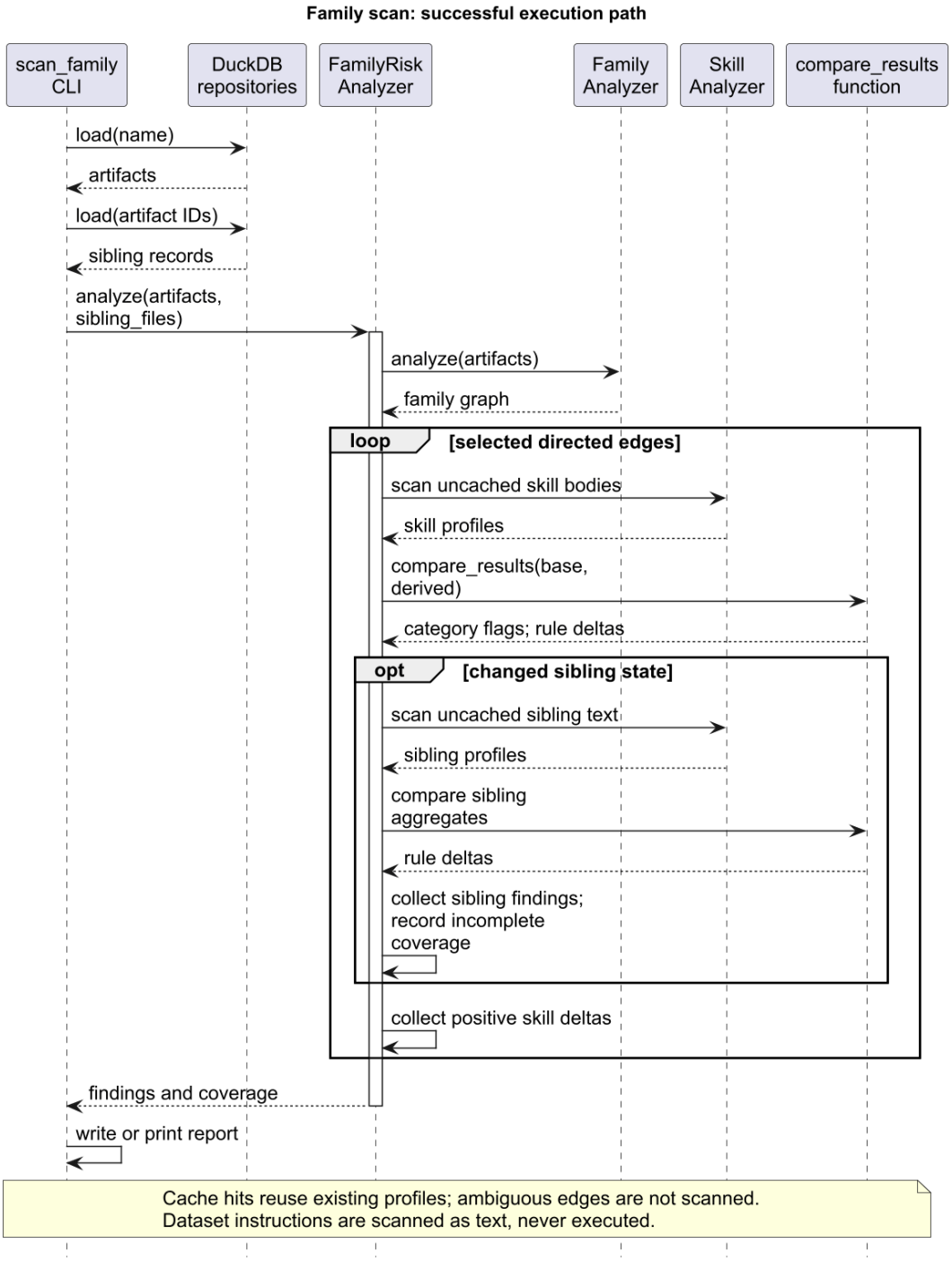


Figure 3: Sequence diagram of family scanning, including conditional sibling analysis and comparison.

9. Produce structured result tables, summary measures, validation inputs, logs, and report-ready figures or tables.

The pipeline records the scanner version, rule-catalog version, configuration, and other run metadata needed to connect every result to the implementation that produced it.

The implemented family workflow is narrower than that census target: `DuckDBFamilyRepository` loads one exact-name family, `FamilyAnalyzer` selects relationships, and `FamilyRiskAnalyzer` scans selected edges using per-analysis caches. Skill scans are reused by skill SHA; sibling scans reuse available content by SHA, with occurrence-based fallback where necessary. This does not establish a persisted, population-wide profile store.

After completing the database preparation documented in `data/README.md`, install dependencies with `uv sync` and use `uv run python -m gitskills.tools.analyze_family --help`, `uv run python -m gitskills.tools.scan_diff --help`, and `uv run python -m gitskills.tools.scan_family --help` for executable options. Family tools accept an exact name and `--db PATH`; `scan_diff` accepts the base and derived paths. An analysis report can be saved to `apps/skilltrace/reports` for SkillTrace. Scan reports must use a separate directory because their schema differs from the graph report schema.

Baselines and Test Fixtures

The earlier or source artifact serves as the within-relationship baseline for each derived artifact. This paired baseline supports a direct claim about whether a signal was detected in the derived artifact but not in its identified base; it is not a general-purpose prediction baseline.

The implementation uses constructed base and derived pairs under `tests/sample_skills`, with expected outputs under `tests/pytests/test_configs`. The shared pytest runner checks introduced categories, total counts, and individual rule deltas. These fixtures establish expected behavior on selected inputs; they are not empirical population observations or a measurement of scanner precision and recall. Additional coverage of removals, unchanged counts with changed text, and difficult direction cases should be evaluated explicitly.

[TODO: Define any additional baseline rule set, heuristic, comparison method, or ablation required by the final evaluation, and explain what question that baseline answers.]

Design Decisions

Ordered Comparisons Rather Than Isolated Artifacts

The research question concerns behavior newly introduced through modification or reuse. An ordered base-derived pair therefore provides a more direct unit of analysis than an isolated artifact.

Rule-Level and Category-Level Measurement

The method reports both levels because category-only comparisons can hide the introduction of a new rule inside an already-present category, while rule-only results may be too granular to summarize clearly.

One Rule Catalog per Comparison and Run

Scanning both sides with the same rule definitions prevents catalog changes from being mistaken for changes in artifact content. A run should identify those definitions by code revision or an explicit catalog version; after a rule change, affected analyses must be rerun before comparison with newer outputs.

SHA-Based Caching Without Statistical Deduplication

Within a family analysis, scanning a skill SHA once avoids redundant computation while preserving the selected artifact relationships. The broader census design should retain artifact and relationship occurrences when reusing profiles, so a cache does not silently redefine the research population.

Census of Eligible Comparisons

The planned census will attempt to analyze every eligible comparison. Current family-level runs and development subsets support testing and exploration; they do not establish completion of the population-level run.

Evidence Retention and Auditable Filtering

The family tools retain relationship bases, direction evidence, rule findings, and incomplete sibling coverage. The broader population pipeline also needs auditable eligibility and exclusion records; those records must be verified before claiming complete population accounting.

Static Analysis and Cautious Interpretation

The project does not execute untrusted artifact content. Detected introductions are reported as static risk signals. They do not, by themselves, prove malicious intent, vulnerability, exploitability, or real-world impact.

Generated Outputs

The broader study is expected to produce:

- an artifact table with traceability fields;
- a relationship table with ordering evidence and eligibility status;
- versioned scanner profiles;
- an ordered comparison table with introduced, removed, and retained signals;
- exclusion counts and reasons;
- aggregate summary tables;
- a seeded manual-validation sample;
- logs and run metadata; and
- report-ready tables and figures generated from structured results.

Currently implemented outputs include family-analysis JSON from `analyze_family`, paired comparison JSON from `scan_diff`, family review JSON from `scan_family`, and optional verbose evidence. These formats serve different purposes and are not interchangeable. A complete population comparison table, calibrated accuracy measures, and census summary figures remain deliverables to verify rather than completed results.

Reproducibility

A reproducible run must identify:

- the dataset release or snapshot and integrity information;
- the required environment and dependency versions;
- configuration and command-line arguments;
- scanner and rule-catalog versions;
- random seeds used for validation sampling;
- executable commands for each pipeline stage and the complete pipeline;

- output locations and expected artifacts; and
- known platform, resource, or data-availability limitations.

[TODO: Add the final setup instructions and exact clean-environment execution command after the implementation is complete.]

Evaluation and Validation

Evaluation Objectives

Evaluation separates three questions: whether extraction preserves the intended inputs, whether relatedness and direction are supported, and whether scanner findings correspond to the operational rule definitions. A correct count on an incorrectly ordered pair is not a valid introduction finding. A plausible relationship does not establish that a rule accurately captures the intended behavior. The broader census and manual-review plans remain distinct from the implemented test suite and illustrative local runs.

Independent Database Reconstruction

Team members independently downloaded `agent_skills_release.db` and executed the project SQL queries on their own machines. The team reports that these independent runs accurately reproduced the results. This is a completed cross-machine reproducibility check of the data-preparation workflow, separate from automated scanner tests and the planned manual validation of lineage and security findings.

This observation supports repeatability in the tested setups. The available team account does not specify a per-field comparison protocol, so it is not presented as a documented exhaustive comparison of every artifact-to-ID mapping. A reproducibility record should preserve the dataset checksum, query revision, DuckDB version, and compared outputs alongside this observation.

Automated Tests

The shared runner `tests/pytests/test_sample_skills.py` discovers JSON expectations under `tests/pytests/test_configs`, loads matching base and derived `SKILL.md` files, and invokes the `scan_diff` entry point. It checks successful completion, introduced-category membership, total base/derived match counts and their delta, and the full nonzero rule-delta mapping. This tests the file-input, scanning, comparison, and JSON-output path together.

A local verification at code revision `f2b4c7d` on October 15, 2026 passed all seven configured sample cases using:

```
uv run pytest tests/pytests/test_sample_skills.py -q
```

This result describes that test selection, not a claim that all project tests passed or that the scanner achieves a population-level accuracy target. Systematic tests for similarity boundaries, provenance conflicts, missing chronology, cycle avoidance, and incomplete sibling coverage are additional validation needs.

Constructed Test Fixtures

Constructed cases isolate recognizable changes rather than reproduce the distribution of real-world artifacts. The configured cases cover command/network introductions, file-system/system-modification introductions, credential introduction, external-code introduction, an unchanged-capability control, frontmatter-only changes, and file-reading behavior. The frontmatter case checks that metadata does not create a behavioral finding; the control checks that expected unchanged behavior remains unchanged.

Expected outputs are assertions about the current rules. Passing them demonstrates regression consistency, not independent proof that the rules capture every intended behavior. Additional

fixtures should cover removal, equal counts with different instructions, overlapping patterns, formatting variants, and unrecognized syntax. Dataset-derived examples must be labeled separately from constructed fixtures.

Sprint 1 Sample as a Tool-Validation Dataset

Purpose and Selection

Sprint 1 used 43 dataset artifacts across ten exact-name families to exercise the developing tools on real collected content. RDR-006 records why the team replaced an earlier roughly 1,400-row mixed selection with this compact purposeful sample: a manageable set could be inspected in context while preserving multiple artifacts per family. Screening sought package-installation variation, supported source-metadata fields, and variation in additional scanner rules. These heuristics selected informative cases; they did not establish lineage or scanner correctness.

The sample complements the constructed fixtures. Fixtures isolate an expected behavior in a small controlled input; the Sprint 1 artifacts expose combinations of metadata, repeated content, missing evidence, and instruction text that occur in the collected dataset. The sample was not randomly drawn and does not support estimates of population-wide risk prevalence.

Two Workflows Exercised Different Parts of the Tools

The family-analysis workflow loaded the selected exact-name families, constructed skill and bundle variants, measured similarity, and attempted direction inference. The ten saved family reports contained 17 selected directed comparisons. Equivalent and ambiguous relationships remained visible without being forced into the directional scan. Selected comparisons were manually inspected in Mergely to compare reported changes with the source text. This was targeted inspection, not independent annotation of every relationship.

The extraction notebook used the same sample for a different integration task. It read `sample_artifacts` in view order, grouped by name while preserving that order, and compared each occurrence with its immediately preceding occurrence in the group. With 43 rows across ten groups, this produces 43 minus 10, or 33 adjacent comparisons: the first row in each group has no preceding row. The notebook then scanned both file bodies, recorded positive rule deltas and complete comparison JSON, wrote Parquet, reloaded it, and generated summary figures.

The 33 notebook pairs are ordering-based extraction comparisons, not the 17 edges selected by the family algorithm. Adjacency in the SQL view does not establish parentage. Their totals and positive-delta rates must therefore be reported separately, and notebook pairs must not be described as independently verified parent-child relationships.

What the Sample Tested

Tool behavior	How Sprint 1 exercised it	What the check can establish
Candidate loading and grouping	Load multiple occurrences per exact name, including description variants	Whether the tools preserve the candidate context needed for comparison
Content and bundle representation	Compare repeated or differing skill states and available sibling fingerprints	Whether skill and bundle state are distinguished; unknown sibling evidence is not proof of equality

Tool behavior	How Sprint 1 exercised it	What the check can establish
Similarity and direction reporting	Examine supported direction, equivalent peers, ambiguity, and cases without directed edges	Whether the tools expose their evidence and avoid forcing every candidate into a parent-child pair
Scanner and comparator output	Inspect positive, negative, or unchanged counts and category-introduction flags	Whether existing signals, count changes, and introduced categories are distinguished on selected cases
Extraction and output integration	Run the notebook from sample view through paired scans, Parquet, and figures	Whether stages can exchange and preserve the expected fields and result structures
Failure investigation	Inspect source diffs and retained formatting/coverage cases	Whether unexpected counts can be traced to inputs or rule behavior and turned into regression checks

The earlier inspections exposed uneven recognition of equivalent operations and artificial count differences associated with line endings. The earlier wrapper was corrected and the formatting case retained for regression checking. The saved Sprint 1 reports showed delta variation in ten of the then-current twelve rules, while EXT-001 and SYS-001 lacked confirmed delta examples in that selection. These observations identify useful coverage and gaps; they do not provide a measured precision or recall for the current thirteen-rule catalog.

Automated Support and Evidence Limits

`tests/pytests/test_extraction_pipeline.py` exercises the notebook's loader, validation, adjacent pairing, positive-delta accounting, and Parquet round trip against a temporary DuckDB fixture. These automated checks complement the dataset-based notebook run without requiring the full dataset. They verify implementation behavior rather than independently validating the sample's inferred ancestry or security significance. A local verification on October 15, 2026 passed all 21 extraction-pipeline test cases, using a writable temporary directory for the fixture database and generated files. The test selection can be run with `uv run pytest tests/pytests/test_extraction_pipeline.py -q`.

The saved notebook output can establish which comparisons were produced and how their scan results were summarized. It does not establish that every pair was manually reviewed, that the current rules match the definitions used in the saved run, or that findings generalize beyond this selected sample. Saved-run outcomes are reported separately in Results. Broader relationship review, scanner error estimation, and the planned population census remain additional evaluation tasks.

Manual Validation Sample

The planned broader review comprises at least 100 eligible comparisons, divided initially between at least 50 with a reported rule/category introduction and at least 50 without one, as described in Dataset and Data Preparation. This review has not been established as completed. Before selection, the team must freeze the run, define relationship/category strata, record a random seed, and document allocation and replacement when a stratum is too small. Count increases within existing rules should be identified separately from introductions when constructing these strata.

Annotation Protocol

The proposed protocol records two separate judgments per comparison. First, reviewers inspect candidate scope, similarity, provenance and chronology evidence, direction basis, and unresolved alternatives. Second, they inspect both file bodies and the relevant match locations, checking whether each reported rule and introduced category is supported. For sibling findings, reviewers also check which destination resources changed and whether either endpoint lacks required content.

Review records should distinguish supported findings, unsupported matches, missed in-scope patterns, uncertain direction, and incomplete inputs. Disagreements should retain both initial judgments and an adjudicated explanation. The number of reviewers, independence procedure, and final annotation form remain to be specified before claiming reviewer agreement.

Validation Measures

For independently reviewed findings, precision is supported positive findings divided by all reviewed positive findings under the operational labeling protocol. Recall requires an independently annotated set of actual in-scope positives, including missed findings; reviewing only scanner hits cannot estimate it. A balanced or stratified review sample does not directly estimate population prevalence without accounting for selection probabilities.

Relationship support and direction agreement should be reported separately from scanner correctness. Ambiguous and incomplete cases require their own counts rather than automatic assignment as negatives. No numerical precision, recall, or reviewer-agreement result is claimed here before the corresponding annotations exist.

Error Analysis

Error analysis should distinguish candidate-generation failures, source-resolution failures, ordering uncertainty, missing content, syntax coverage gaps, documentation-only matches, and formatting sensitivity. Fixes must be checked against frozen regression cases and affected analyses rerun. The earlier Sprint 1 inspections motivated a line-ending correction; that historical observation does not establish that all formatting or rule-coverage issues have been resolved.

Risk-Signal Interpretation Disclaimer

A detected risk signal indicates that a skill contains an instruction pattern identified as security-relevant by the project's current rule catalog. It does not prove that the skill is malicious, vulnerable, exploitable, or unsafe.

Likewise, an introduced risk signal means that the scanner detected a rule or category in the derived artifact that it did not detect in the corresponding base artifact. This is a comparison of static artifact content, not evidence that the instruction was executed or caused harm.

The available evidence may not establish:

- whether the instruction is executable in the intended environment;
- whether an agent or user ever executed it;
- whether execution requires user approval or additional permissions;
- whether security controls would prevent or restrict the behavior;
- whether a referenced file, script, command, credential, or network resource is harmful;
- whether the behavior creates an exploitable condition; or
- whether the artifact's author intended malicious behavior.

Many legitimate skills require commands, file operations, network access, scripts, or authentication instructions to perform their intended functions. The project will therefore describe scanner findings as **security-relevant capabilities**, **risk signals**, or **patterns requiring review**. Terms such as **malicious**, **vulnerable**, and **exploitable** will be used only when supported by additional evidence beyond a static rule match.

Results

This section distinguishes saved Sprint 1 observations, a locally reproduced methodological example, and population-level results that remain to be produced. The worked example demonstrates tool behavior; it does not estimate prevalence or validate historical ancestry.

Dataset Processing Results

The local candidate-selection audit on October 15, 2026 retained 943,257 of 3,797,117 source artifact occurrences, a reduction of 75.16 percent. The retained rows represent 162,995 exact-name families and 628,026 name-and-normalized-description groups. Dataset and Data Preparation provides the executable SQL, cumulative counts, and filter rationale. These totals describe candidate construction, not completed similarity analyses, directed comparisons, or security scans. Unique analyzed SHA counts, final relationship totals, scan completion and exclusion counts remain to be measured by the broader pipeline.

Saved Sprint 1 Extraction Outcomes

Inspection of `notebooks/extraction_pipeline/results/sample_artifacts_scanner_results.parquet` confirmed the following saved results. This is an audit of the saved output, not a rerun of every dataset comparison with the current rules.

Saved notebook measure	Observed value
Artifact rows	43
Exact-name groups	10
Adjacent comparisons with stored scanner output	33
Comparisons with at least one positive rule-count delta	13 (39.4%)
Positive rule-delta events across those comparisons	35
Sum of positive rule-count deltas	77

The 77 additional matches sum positive deltas only; they are not the net change after accounting for decreases, a count of unique actions, or a count of newly introduced capabilities. Multiple rules can increase in one pair, and multiple matches can contribute to one rule's delta.

The 33 notebook pairs follow view-order adjacency within names, whereas the saved family-analysis reports contain 17 selected directed relationships. These outputs test different parts of the workflow and must not be combined into one relationship denominator. The 39.4 percent figure describes this purposeful notebook sample only, not population prevalence or a rate of proven harmful changes.

Security-Pattern Results

[TODO: Report the prevalence of detected rules and categories.]

Introduced Risk Signals

[TODO: Report how frequently derived artifacts introduced rules or categories that were absent from their base artifacts.]

Removed and Retained Signals

[TODO: Report how frequently signals were removed, retained, or unchanged.]

Results by Relationship Type

[TODO: Compare meaningful relationship groups when supported by the data.]

Validation Results

The configured sample-skill test selection passed seven cases at the documented code revision, and a separate local verification passed all 21 extraction-pipeline test cases. These deterministic checks establish expected outputs for those fixtures; they do not estimate accuracy on dataset artifacts. The worked example below was reproduced from local data, but neither its inferred direction nor its scanner findings have been established here as independently annotated ground truth. Numerical manual-review results remain pending the review protocol and annotations described in Evaluation and Validation.

Representative Examples

Worked Example: Similarity, Inferred Direction, and Scanner Change

The example uses one five-artifact candidate family from the prepared local DuckDB database. Family and endpoint identities are omitted under the report's reporting policy; the selected source and target are labeled A and B. The analysis was rerun locally on October 15, 2026 at code revision `f2b4c7d`, using the default similarity policy and current registered scanner rules. The database release identifier and integrity hash have not yet been documented, so this is a local demonstration rather than a fully frozen dataset experiment.

From Candidate Family to Selected Pair The family contains five skill variants and five bundle variants in one similarity cluster. All ten distinct skill-variant pairs pass the relatedness policy, but the resulting graph selects only two directed predecessor edges. It also records one equivalent undirected relationship and two direction-unknown relationships; other supported directed candidates are retained as additional relationships.

For the selected A-to-B edge, the analysis reports 2,936 shared shingles, containment of approximately 0.962, and Jaccard of approximately 0.414. Thus, about 96.2 percent of A's distinct shingles occur in B, while their shared shingles account for about 41.4 percent of the combined set. These are representation-overlap measures, not percentages of lines copied or probabilities of common ancestry.

The selected edge has `containment` as its direction basis and is classified as `skill-only`. This supplies a structural hypothesis for treating A as base and B as derived; it does not establish that A was published first or directly copied. The second selected edge in the family uses chronology, illustrating that one family can contain relationships with different evidential bases.

```
Five exact-name artifact occurrences
          |
Five skill variants; ten qualifying similarity pairs
          |
One cluster; two selected directed predecessor edges
          |
Selected pair A -> B (containment-based, skill-only)
          |
Scan both skill bodies with the same 13-rule catalog
          |
25 base matches -> 49 derived matches; no new categories
```

Paired Scanner Results

Changed rule	Base A	Derived B	Delta
CMD-001: shell-language code fences	3	5	+2
CMD-002: command invocations	2	6	+4
CRED-001: secret-related terms	11	14	+3
CRED-002: credential-file references	5	12	+7
EXT-002: package installation	1	3	+2
FS-003: file-reading commands/APIs	1	2	+1
NET-001: HTTP/HTTPS URLs	2	7	+5
Total rule matches	**25**	**49**	**+24**

Each changed rule already matches the base, so none of these increases constitutes new rule presence. All introduced-category flags are false. This example therefore demonstrates increased matches within existing capabilities rather than the introduction of a previously absent category. It cannot support a claim that the derived artifact is more harmful, that instructions executed, or that any credential was exposed.

These counts describe the current code revision and local inputs. Counts depend on the rule catalog, input handling, and analyzed text. Runs using different definitions or processing settings must be labeled separately rather than pooled as measurements from an identical instrument.

Scan Scope and Review Implications This table compares the two skill bodies using `SkillAnalyzer` and `compare_results`, the same scanning and comparison components used by `scan_diff`. It does not present sibling findings. The broader `scan_family` path reviews selected directed edges and applicable sibling changes, and can flag matches in new destination content even without a positive aggregate count delta.

A reviewer should inspect the matched instructions in context, assess the direction evidence separately, and distinguish missing sibling evidence from a negative result. This case shows that the pipeline can connect candidate selection, inferred direction, and rule-level differences; it supplies neither population-level generalization nor independent ground truth.

Results Summary

The reproduced family example contains five artifacts, ten qualifying similarity pairs, and two selected directed relationships. Its selected containment-based pair has 25 base matches and 49 derived matches, with increases in seven rules and no introduced categories. Seven configured sample-skill tests pass. These observations demonstrate selected implementation behavior; population introduction rates and independently measured validation accuracy remain unreported.

Discussion

Interpretation of Findings

[TODO: Explain what the results suggest about security-relevant changes in modified or reused GitSkills.]

Comparison With Expectations

[TODO: Compare the findings with the expectations stated earlier in the report.]

Simpler and Competing Explanations

[TODO: Discuss plausible explanations other than insecure or malicious propagation, such as legitimate operational requirements, artifact size, shared templates, or scanner bias.]

Security and Supply-Chain Implications

[TODO: Discuss the implications for skill authors, reviewers, tool developers, repositories, and users.]

Practical Recommendations

[TODO: Present recommendations supported by the findings.]

Unexpected Findings

[TODO: Discuss surprising results and possible explanations.]

Interpretation Boundaries

[TODO: Reinforce what the evidence does and does not establish, including limits involving intent, execution, exploitability, causation, and provenance.]

Threats to Validity

Construct Validity

The study uses rule-based indicators, candidate groupings, similarity measures, provenance evidence, and sibling-content fingerprints as operational representations of security-relevant instructions and artifact relationships. These constructs do not directly establish behavior during execution, maliciousness, or historical copying.

Rule presence, changes in match counts, and newly detected capabilities are distinct outcomes. A count increase can occur within an already detected capability, while a negative delta means fewer detected matches rather than necessarily lower risk. A missing entry in `rule_deltas` does not establish that a rule never matched. SQL screening patterns approximate scanner behavior and are selection heuristics rather than security findings.

`artifact_groupings` groups artifacts by exact name and normalized description, while `analyze_family` loads qualifying artifacts across description groups sharing an exact name. Neither shared metadata nor high similarity proves common ancestry. Similarity measures establish candidate relatedness; resolved declared provenance, chronology, and asymmetric containment provide direction evidence. Similarity-only relationships retain uncertainty about direction.

A sibling-content fingerprint indicates differences in bundled file content when composition data is available, not their security significance. The saved Sprint 1 wrapper reports scanned only `SKILL.md`. The current family scanner also reviews available sibling content, while `scan_diff` and SkillTrace pair scans compare skill bodies. Findings from these surfaces must be labeled separately; an absent delta on one surface does not establish safety on another.

Internal Validity

Errors in candidate generation, similarity analysis, provenance interpretation, ordering, extraction, normalization, or rule matching can affect the reported relationships and scanner changes. Exact-name loading can group independently developed skills together and miss related skills whose names differ. Screening within description groups can also miss variation across those groups.

Declared provenance, commit dates, and asymmetric containment provide direction evidence but do not guarantee original authorship or direct descent. Commit history follows the artifact at its current path, so moves or renames may obscure earlier history. Supported provenance fields must resolve within the loaded exact-name family. An unsuccessful source lookup does not establish that the source is absent from the entire dataset, and a pair that also fails similarity criteria does not isolate a source-resolution failure.

Selected manual inspections identified uneven recognition of equivalent operation types and artificial count differences caused by line-ending formats. The earlier wrapper normalized line endings before scanning and retained the corrected case for regression checking. Normalization must be verified for each current input path before treating its counts as interchangeable. The historical fix addresses the observed issue, not every possible formatting sensitivity or scanner coverage gap.

Only selected comparisons were manually inspected. Saved reports are automated output rather than independent ground truth, and the sample does not establish scanner accuracy or complete validation of inferred relationships.

External Validity

The revised Sprint 1 sample contains 43 artifacts across 10 exact-name families and produces 17 directed comparisons in the saved reports. Selection was purposeful and aimed to keep the sample below approximately 50 artifacts for focused tool validation. It has no separate random component.

Cases were chosen for informative relationship outcomes, scanner-count changes, newly detected capabilities, and known limitations. This intentionally favors useful validation cases. The earlier approximately 1,400-row design included random seeds and family expansion, but that component does not make the revised sample representative.

Searching stopped because additional screening was time-consuming, not because coverage was exhaustive or statistically optimized. Results describe the method's behavior on selected cases and cannot estimate population-wide risk prevalence or generalize automatically to other repositories, platforms, or datasets.

Conclusion Validity

Security-relevant instructions can be legitimate parts of development, deployment, or administration. Detected matches and changes in their counts do not by themselves establish maliciousness, exploitability, or increased or decreased risk.

The 17 directed comparisons are not independent observations: relationships may share endpoints or repeated content. Small category counts and deliberately selected cases limit quantitative conclusions. The saved Sprint 1 reports demonstrated count deltas in 10 of the then-current 12 scanner rules, with no confirmed delta examples for EXT-001 and SYS-001 in that selection. The current catalog contains 13 rules. Historical coverage and counts cannot establish coverage or accuracy for the revised catalog; reruns must identify their code revision and scan scope.

Ambiguous relationships are listed without directional scanning. A family with zero directed comparisons therefore provides no basis for concluding that its artifacts contain zero scanner matches. Similarly, unchanged aggregate counts can conceal differences in matched text and should not be interpreted as identical behavior.

Reproducibility Threats

Team members independently downloaded the source database and ran the SQL queries on their own machines, reporting accurate reproduction of the results. This is positive empirical evidence of repeatability for those runs. It does not remove the following limitations concerning build-local identifiers, snapshot identity, and verification scope.

The saved Sprint 1 selection uses 43 build-local artifact IDs in `sql/create_sprint1_samples_view.sql`, which creates or replaces `sample_artifacts`. Both artifact and group numbers originate from unordered `row_number()` expressions, so independently rebuilding the same source does not guarantee identical numeric assignments. Reopening the same saved database preserves the existing mapping. The ID list in `data/sample-population/sample_population.csv` is a separate record; the view does not read it. These representations could diverge unless checked together.

The view stores a query rather than a snapshot of artifact rows. Returned data depends on candidate membership and the artifact/repository joins. A remapped numeric ID can select a different source occurrence rather than simply producing a missing row. Reproduction across builds therefore requires the dataset snapshot and an ID-to-source mapping based on `(repo_full_name, path)`; subgroup reconciliation uses exact name and normalized description. Content hashes provide an additional integrity check but do not uniquely identify occurrences. Dataset identity, dependencies, analysis code, rule definitions, and normalization settings can all affect reproducibility.

The script includes a count query with an expected result of 43, but the saved family-report totals do not verify execution or reconstruction of the view. Validation should check every selected ID and compare membership in both directions; a matching count alone is insufficient. Earlier frozen-table reconstruction checks apply to the earlier sample, not to this revised view.

Missing and Incomplete Data

Missing content, chronology, or sibling-composition data can restrict analysis. Missing values do not establish the absence of a behavior or resource; they may reflect incomplete enrichment. Artifact pairs lacking adequate direction evidence remain ambiguous rather than being forced into an ordered comparison.

Commit history may be missing or limited to the current path. Sibling-composition coverage is uneven across repeated content occurrences, limiting comparisons of bundled resources. The current family scanner can scan available sibling content and records unavailable changed resources or unknown SHA evidence as incomplete coverage. This extends the older skill-only workflow but does not eliminate missing-data bias. Content recovered through a matching SHA should retain its evidence source, and missing content must not be interpreted as absence of behavior.

Filtering for available content, relatedness, or supported direction can bias results toward better-enriched records. Exclusions and ambiguity should therefore be reported separately from negative scanner findings. Candidate construction additionally requires valid frontmatter, a body longer than 50 characters, non-null name and description, and at least two qualifying occurrences with the same exact name. This excludes malformed or very short skills, metadata-poor artifacts, and singleton names before similarity analysis. The 50-character threshold is a heuristic rather than a validated quality boundary. Any later prevalence estimate must state whether its denominator is the source population, the 943,257 retained candidates in the local audit, or the further restricted set of supported directed comparisons.

Output Scope and Traceability

Reports use artifact IDs, omit text diffs, and serialize scanner comparisons with `verbose=False` to limit exported content. This report describes aggregate results and generalized observations without selected family names, individual artifact IDs, repository identities, or source excerpts.

These measures do not guarantee anonymity or resolve licensing questions. Someone with the dataset can link an artifact ID to its source record, and the local sample view exposes all columns from matching artifacts. Limited exported evidence also means that detailed verification requires access to the corresponding local data and analysis settings.

Mitigations

- Distinguish candidate relatedness from direction evidence, and preserve ambiguous relationships without directional scanning.
- Separate selection heuristics, observed tool output, selected manual inspection, and unresolved limitations.
- Treat scanner matches as signals rather than proof of risk, and report count changes separately from introduced capabilities.
- Preserve exact-name family context while documenting candidate-generation and provenance-resolution limits.
- Recheck the historical line-ending regression across current input paths, and reassess rule-delta coverage against the current 13-rule catalog.

- Version the embedded artifact-ID selection, reconcile it with the CSV list, and verify the ID-to-source mapping before reuse in another database build.
- Verify membership and dataset identity before claiming successful reconstruction of the revised sample.
- Record incomplete inputs and distinguish unscanned cases from negative findings.
- Keep broader census and stratified manual-review plans separate from completed Sprint 1 validation.
- Record methodological decisions and output constraints in research decision records and report documentation.

Related Work and References

Mining Software Repositories

[TODO: Summarize relevant research involving repository mining, artifact evolution, reuse, and software change analysis.]

Software Supply-Chain Security

[TODO: Summarize relevant work involving reused artifacts, dependency risk, provenance, and supply-chain propagation.]

Static Security Analysis

[TODO: Summarize relevant work involving rule-based scanning, command detection, configuration analysis, or security-pattern identification.]

AI-Agent Skills and Instructions

[TODO: Summarize relevant research or documentation involving AI-agent skills, prompt-based tools, agent instructions, and AI-native software-engineering artifacts.]

Relationship to This Project

[TODO: Explain how this project differs from or extends the related work.]

References

Conclusion and Future Work

Conclusion

[TODO: Restate the research question, summarize the method, and present the most important findings.]

Contributions

[TODO: Summarize the project's technical, empirical, and methodological contributions.]

Limitations

[TODO: Briefly acknowledge the limitations most important to interpreting the conclusion.]

Future Work

[TODO: Identify reasonable extensions, such as additional rules, improved relationship detection, broader validation, or analysis of other datasets.]

Final Takeaway

[TODO: End with the principal implication for AI-agent skill development, review, or security.]

Artifact Appendix

Repository

[TODO: Provide the repository URL, release tag, commit identifier, and access information.]

Project Structure

[TODO: Describe the important source, test, data, results, figure, documentation, and report directories.]

Environment and Dependencies

[TODO: List the supported operating environment, language version, dependencies, and installation instructions.]

Dataset Availability

[TODO: Explain how to obtain `agent_skills_release.db`, where it should be placed, and why it is not included in the repository.]

Reproduction Commands

[TODO: Provide the exact commands required to run tests, extract data, execute the scanner, perform comparisons, generate results, and build the report.]

Generated Artifacts

[TODO: List the primary result tables, figures, logs, and report outputs and identify which commands generate them.]

Configuration and Versioning

[TODO: Document the scanner version, rule-catalog version, configuration files, random seeds, and other settings associated with the reported results.]

Known Limitations

[TODO: Describe practical limitations affecting installation, execution time, storage, hardware requirements, or supported inputs.]

AI-Use Disclosure

[TODO: Summarize how AI tools were used, how generated work was reviewed, and where the detailed AI-use log can be found.]

Team Contributions

[TODO: Summarize each team member's documented technical, research, review, and project-management contributions.]